

WHAT IT IS

Independent Bengali publishing storefront built with Next.js App Router. It sells and catalogs books, publishes journal/author pages, and supports contact and manuscript-submission flows backed by Prisma/PostgreSQL.

WHO IT'S FOR

Primary: Bengali literature readers and book buyers. Secondary repo-supported users: writers submitting manuscripts, customer account holders, and staff admins.

WHAT IT DOES

- Public storefront for home, books, authors, blog/journal, about, contact, manuscript, policy, robots, and sitemap routes.
- Catalog search, genre and price filters, sorting, pagination, book detail pages, stock status, and add-to-cart UI.
- Local-storage cart and guest checkout with delivery details, shipping quote, and COD-first payment flow.
- Server-side order creation with authoritative pricing, rate limits, stock checks, and inventory commit/release.
- Optional Razorpay online payment with checkout verification, webhook handling, and failed-payment handling.
- Login/register/account area with signed sessions and guest-order linking by customer email.
- Admin workspace for catalog, authors, blog, orders, inbox, newsletter, settings, launch readiness, Cloudinary image upload, and seed import.

HOW IT WORKS

- App layer: Next.js 16 App Router under app/; React components under components/; Tailwind CSS UI and public chrome in layout components.
- Content flow: app pages call lib/content; it reads Prisma/PostgreSQL and falls back to seed catalog arrays when DB content is absent.
- Commerce flow: CartProvider stores items in localStorage; CheckoutClient posts to /api/checkout/create-order; API validates with Zod/rate limits, recalculates totals, writes Order/OrderItem records, and updates inventory.
- Payment/email flow: Razorpay routes call lib/payments for HMAC checks and status changes; EmailJob rows queue Resend emails after response or through /api/internal/email-jobs/process.
- Admin flow: /admin server actions require admin role, mutate Prisma models, revalidate storefront paths, and use Cloudinary for configured image uploads.

HOW TO RUN

1. npm install
2. Copy .env.example to .env.local and fill required values.
3. npm run db:local:setup
4. npm run prisma:push
5. npm run content:bootstrap
6. npm run admin:create -- editor@example.com strong-password "Admin Name"
7. npm run dev; open http://localhost:3000